

Estudiantes (Pareja): Guerra y Solórzano | Docente: Guillermo Calderón | Entorno: ParrotOS (host local) · Linux x86\_64 · cgroups v2

Investigación 1. PID namespace y punto de observación (10 puntos)

**Pregunta:** Determinar cómo cambia el identificador visible de un proceso cuando se observa desde un PID namespace y desde el host.

Comandos ejecutados (dos vistas):

```
# Vista dentro (equiv. Terminal A)
sudo unshare --pid --fork --mount-proc bash -c 'printf "PID visto dentro: %s" "$$"; sleep 120'

# Vista host (equiv. Terminal B) - filtrada para no volcar los procesos del host
ps -eo pid,ppid,comm,args --forest | grep -E "PID|unshare|sleep 120"
grep '^NSpid:' /proc/824974/status
grep '^NSpid:' /proc/824975/status
```

Tabla de evidencia observada:

| Dato                                       | Valor observado | Fuente                                       |
|--------------------------------------------|-----------------|----------------------------------------------|
| PID visto dentro del namespace (Captura 1) | 1               | salida del shell dentro del namespace (\$\$) |
| PID del mismo proceso (bash) desde el host | 824974          | ps --forest filtrado                         |
| NSpid: del bash del namespace              | NSpid: 824974 1 | /proc/824974/status                          |
| NSpid: del sleep 120 interno               | NSpid: 824975 2 | /proc/824975/status                          |

```
## INVESTIGACION 1 - PID namespace y punto de observación
Se reproducen las dos vistas (dentro del namespace y del host) en esta misma terminal. Para replicar el flujo original en dos terminales:
Term A: sudo unshare --pid --fork --mount-proc bash (luego printf ...; sleep 120)
Term B: ps -eo pid,ppid,comm,args --forest | grep unshare|sleep 120 (filtrado)

>>>> CAPTURA 1 - Vista desde dentro del namespace (equivalente Terminal A) <<<<

$ sudo unshare --pid --fork --mount-proc bash -c 'printf "PID visto dentro: %s\n" "$$"; sleep 120' &
(esperando que el proceso arranque; si sudo pide password, ingresala)
PID visto dentro: 1
(el bash del namespace tiene PID host=824974; su 'sleep' interno PID host=824975)
```

Captura obligatoria 1: Vista dentro del PID namespace: el proceso se imprime como PID 1.

```
>>>> CAPTURA 2 - Vista desde el host (jerarquía filtrada y NSpid) <<<<
Ej: host=parrot. Solo se muestran las líneas del unshare/bash/sleep.

$ ps -eo pid,ppid,comm,args --forest | grep -E "PID|unshare|sleep 120"
PID      PPID  COMMAND      COMMAND
824966   824557      sud          \ sudo unshare --pid --fork --mount-proc bash -c printf "PID visto dentro: %s\n" "$$"; sleep 120
824973   824966      sud          | \ sudo unshare --pid --fork --mount-proc bash -c printf "PID visto dentro: %s\n" "$$"; sleep 120
824974   824973      uns          | | unshare --pid --fork --mount-proc bash -c printf "PID visto dentro: %s\n" "$$"; sleep 120
824975   824974      sle          | | \ sleep 120
826252   824557      gre          | \ grep -E PID|unshare|sleep 120

$ grep ""NSpid:" /proc/824974/status
NSpid: 824974
$ grep ""NSpid:" /proc/824975/status
NSpid: 824975 1

Interpretación rápida: el mismo proceso se ve como PID 1 dentro del namespace y como PID 824974 desde el host (la línea NSpid lista ambos).
```

Captura obligatoria 2: Vista desde el host: jerarquía filtrada del unshare/bash/sleep y líneas NSpid de los procesos 824974 y 824975.

1.1. ¿Qué PID observa el proceso desde dentro del namespace?

El PID 1: es el primer proceso del nuevo PID namespace creado por unshare --pid --fork, por lo que la numeración reinicia desde 1 y le corresponde el 1 (así lo imprimió el \$\$ del shell).

1.2. ¿Qué PID corresponde al mismo proceso desde el host? Interprete la línea NSpid.

Desde el host, el mismo proceso tiene el PID 824974. La línea NSpid lista los identificadores del proceso en cada nivel de la jerarquía de PID namespaces, del más externo (host) al más interno: NSpid: <pid\_host> <pid\_ns>. Así, NSpid: 824974 1 verifica que el "1" visto dentro corresponde al PID 824974 observado desde el host; NSpid: 824975 2 hace lo propio con el sleep (PID 2 dentro del namespace).

### 1.3. ¿Por qué dos identificadores distintos no implican dos procesos independientes?

Porque el PID no es una propiedad intrínseca del proceso sino una etiqueta **relativa al namespace** en el que se muestra. Es el mismo task: la línea NSpid del **mismo** `/proc/824974/status` exhibe ambos números para una única tarea, sin que exista ninguna "copia" por namespace.

### 1.4. ¿Qué cambia en la visibilidad de los PID? ¿Qué no permite concluir este experimento sobre CPU y memoria?

Cambia la **visibilidad**: dentro del namespace solo se ven los procesos de ese namespace y sus descendientes, renumerados desde 1 (con `--mount-proc`, `/proc` refleja esa vista); desde el host se ven todos los procesos con su PID global. Este experimento **no permite concluir nada** sobre cuánto CPU o memoria puede usar el proceso: los namespaces cambian la vista, no asignan ni limitan recursos (no se observó ningún contador de uso ni límite).

## Investigación 2. memory.high y presión de memoria (20 puntos)

**Condición experimental:** grupo mem-high con `memory.high=128` MiB y `memory.max=512` MiB; el workload intenta 256 MiB.

| Dato        | Valor observado                                  |
|-------------|--------------------------------------------------|
| memory.high | 134217728 (128 MiB)                              |
| memory.max  | 536870912 (512 MiB)                              |
| Workload    | ./run-in-cgroup.sh mem-high ./mem_work 256 100 5 |

**Predicción:** espero que el proceso **no sea terminado** y que **sí pueda superar** 128 MiB, porque `memory.high` es un límite suave (de presión/reclaim), no un tope duro, y 256 MiB está muy por debajo de `memory.max` (512 MiB). Espero que el contador **high** aumente (eventos de presión al cruzar el umbral), que **oom\_kill se mantenga en 0** y que el workload imprima `allocation complete allocated_mib=256` y `completed normally`, con `exit=0`.

### Evidencia (Captura 3) — contadores `memory.events`:

| Contador | Inicial | Final | $\Delta$ (final – inicial) |
|----------|---------|-------|----------------------------|
| high     | 0       | 125   | <b>125</b>                 |
| max      | 0       | 0     | 0                          |
| oom      | 0       | 0     | 0                          |
| oom_kill | 0       | 0     | 0                          |

**Datos adicionales:**

| Dato                           | Valor observado                                 |
|--------------------------------|-------------------------------------------------|
| memory.current final           | 12288 (grupo sin procesos al leer)              |
| memory.peak                    | 136286208 (≈ 130 MiB, apenas sobre memory.high) |
| ¿Apareció allocation_complete? | Sí (allocated_mib=256)                          |
| ¿Apareció completed normally?  | Sí                                              |
| Código de salida               | 0                                               |

Nota: el pico quedó *cerca* del umbral porque `memory.high` hace *reclaim* continuo: el proceso logró "asignar" (fault) los 256 MiB, pero al superar el high el kernel reclamaba páginas, manteniendo la memoria residente del cgroup alrededor de 128 MiB.

```
--== CAPTURA 3 [pre] - Configuración y contadores utilizados =====  
$ cat /sys/fs/cgroup/memetics-lab02/nom-high/memory.high  
13471728  
  
$ cat /sys/fs/cgroup/memetics-lab02/nom-high/memory.max  
136979212  
  
$ cat /sys/fs/cgroup/memetics-lab02/nom-high/memory.current  
0  
  
$ cat /sys/fs/cgroup/memetics-lab02/nom-high/memory.events  
high 0  
low 0  
oom_kill 0  
oom_group_kill 0  
  
--== PASADOCERO: antes de continuar, lo que esperas que ocurra al  
superar 128 MiB (iteración) supera el valor qe contadores cuentan).  
  
allocated_mib-256  
allocated_mib-256  
allocated_mib-256  
allocated_mib-256  
allocated_mib-256  
allocated_mib-256  
allocated_mib-256  
allocated_mib-256  
allocated_mib-256  
allocated_mib-256  
allocated_mib-256  
allocated_mib-256  
allocated_mib-256  
allocation_complete_allocated_mib-256  
allocation_start_pending-0  
holding_seconds_pending-0  
holding_seconds_pending-0  
holding_seconds_pending-0  
holding_seconds_pending-0  
holding_seconds_pending-0  
compleated normally  
  
exit=0  
  
==== CAPTURA 3 (post) - Contadores finales de memory.events ====  
$ cat /sys/fs/cgroup/memetics-lab02/nom-high/memory.events  
low 0  
high 0  
oom 0  
oom_kill 0  
oom_group_kill 0  
  
Use final de memoria del grupo:  
  
$ cat /sys/fs/cgroup/memetics-lab02/nom-high/memory.current  
128M  
  
$ cat /sys/fs/cgroup/memetics-lab02/nom-high/memory.peak  
13471728  
  
Incrementos {final - inicial}  
high          final+255   0+255  
max           final+0     0+0  
min           final+0     0+0  
oom_kill      final+0     0+0
```

**Captura obligatoria 3:** Inicial (izq.): configuración y contadores de *mem-high*. Final (der.): terminación normal de *mem\_work* (*allocation complete, completed normally, exit=0*) y *memory.events* con *high 125* y *oom kill 0*.

### 2.1. Contraste la predicción con el resultado observado.

La predicción se cumplió: el workload superó los 128 MiB (llegó a `allocated_mib=256` con `allocation_complete`), terminó con `exit=0` y `completed normally`; el único contador relevante que subió fue `high` ( $\Delta = 125$ ) y `oom kill` quedó

en 0. Comportamiento típico de un límite blando.

## 2.2. ¿Qué evidencia permite afirmar que `memory.high` no actuó como un límite duro de 128 MiB?

- `allocation_complete allocated_mib=256` y `completed normally` con `exit=0`: el proceso completó su asignación y no fue interrumpido.
- `memory.peak` (136286208  $\approx$  130 MiB) supera `memory.high` y además el contador `high` subió 125 veces: hubo cruces del umbral **sin matar** al proceso (`oom_kill = 0`). Un límite duro habría frenado o terminado el proceso en 128 MiB.

## 2.3. ¿Qué significado tiene el incremento en el contador `high`? Relaciónelo con la condición experimental.

El contador `high` cuenta cuántas veces el cgroup cruzó/rozó `memory.high` (eventos de presión de memoria). Su  $\Delta = 125$  es el número de eventos durante una carga de 256 MiB frente a un umbral de 128 MiB: mientras el uso quedó por encima del umbral, el kernel aceleró el *reclaim* y pudo *throttlear* al proceso, sin matarlo. La magnitud está acoplada a la condición experimental (carga  $\approx 2 \times$  el umbral).

## 2.4. ¿Puede concluirse, solo porque terminó normalmente, que `memory.high` no tuvo efecto?

**No.** Que haya terminado normalmente no implica "sin efecto": el  $\Delta$  de `high` ( $= 125$ ) demuestra que hubo presión de memoria, asociada al *reclaim* continuo (`memory.peak` quedó pegado al umbral). Lo que **sí** permite concluir es que el límite fue no letal ni bloqueante y que el workload pudo asignar toda su memoria. Lo que **no** permite: cuantificar cuánto se retardó la ejecución (*throttle*) ni el volumen exacto de *reclaim*, porque no se midió tiempo de asignación con/sin *throttle*.

Investigación 3. memory.max y OOM (25 puntos)

**Condición experimental:** grupo mem-max con memory.max=128 MiB, memory.swap.max=0 y sin umbral restrictivo en memory.high; el workload vuelve a intentar 256 MiB.

| Dato            | Valor observado     |
|-----------------|---------------------|
| memory.max      | 134217728 (128 MiB) |
| memory.swap.max | 0                   |
| memory.high     | max                 |

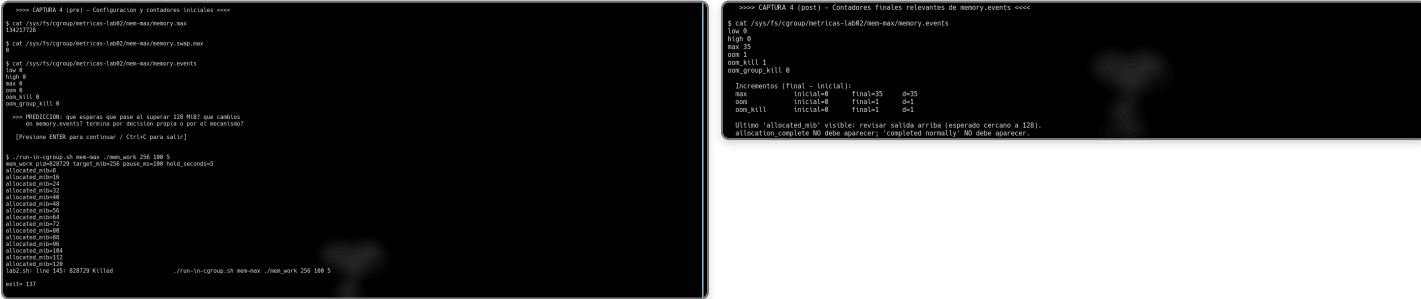
**Predicción:** al intentar superar los 128 MiB espero que el kernel active el *memcg OOM killer* y termine el proceso bruscamente con **SIGKILL**, sin que pueda imprimir `allocation_complete` ni `completed normally` (no termina "por sí solo": la decisión la toma el mecanismo de control de memoria). Espero en `memory.events: max  $\Delta > 0$ , oom  $\Delta \geq 1$  y oom_kill  $\Delta = 1$` , con código de salida 137 (= 128 + SIGKILL).

Evidencia (Captura 4) — contadores memory.events:

| Contador | Inicial | Final | $\Delta$ |
|----------|---------|-------|----------|
| max      | 0       | 35    | 35       |
| oom      | 0       | 1     | 1        |
| oom_kill | 0       | 1     | 1        |

Datos adicionales:

| Dato                           | Valor observado     |
|--------------------------------|---------------------|
| memory.max                     | 134217728 (128 MiB) |
| memory.swap.max                | 0                   |
| Último allocated_mib visible   | 120                 |
| ¿Apareció allocation_complete? | No                  |
| ¿Apareció completed normally?  | No                  |
| Código de salida observado     | 137                 |



Captura obligatoria 4: Inicial (izq.): configuración, contadores en 0 y comienzo de la ejecución. Final (der.): terminación por OOM (proceso Killed, exit=137) y memory.events final con max 35, oom 1, oom\_kill 1.

3.1. ¿Completó el workload la asignación de 256 MiB? Sustente.

**No.** La última línea visible fue `allocated_mib=120` ( $\approx$  `memory.max`), seguida de la notificación de muerte del shell (`lab2.sh: line 145: 828729 Killed`); no aparecen `allocation_complete` ni `completed normally`: el proceso fue interrumpido antes de terminar.

### 3.2. ¿Qué evidencia atribuye la terminación a un OOM del cgroup y no a un error de la aplicación?

- `memory.events` con  $\Delta \text{oom} = 1$  y  $\Delta \text{oom\_kill} = 1$ : el OOM killer del cgroup resolvió el evento matando un proceso del propio cgroup.
- Código de salida `137 = 128 + SIGKILL`: muerte por señal del kernel, no por `return/exit` propio.
- `mem_work` es un asignador simple: un fallo interno habría impreso su propio mensaje y salido con otro código; aquí la app no reportó ningún error.

### 3.3. Función de `max`, `oom` y `oom_kill`. Use sus incrementos.

`max` cuenta los cruces de `memory.max`: su  $\Delta = 35$  indica que el cgroup golpeó el tope de 128 MiB de forma repetida. `oom` cuenta las condiciones OOM del cgroup: su  $\Delta = 1$  señala al menos un estado de falta de memoria crítica. `oom_kill` cuenta procesos terminados por el OOM killer: su  $\Delta = 1$  confirma que se mató **un** proceso (el propio workload, único en el cgroup). Usar los  $\Delta$  (no solo los finales) vincula cada suceso a **esta** ejecución: iniciales en 0 que avanzaron durante el run.

### 3.4. Compare con la Investigación 2.

Con **memory.high**, superar el umbral es permitido: presión y *reclaim*, el proceso sobrevive y termina (`exit=0`,  $\Delta \text{oom\_kill}=0$ ). Con **memory.max**, alcanzar el tope es prohibitivo: sin swap, dispara el OOM killer, que mata el proceso (`exit=137`,  $\Delta \text{oom\_kill}=1$ ). *high* = límite blando (presión/reclaim); *max* = límite duro (terminación).

## Investigación 4. cpu.max y throttling (25 puntos)

**Condición:** cpu-half con `cpu.max = 50000 100000` (50 ms de CPU por período de 100 ms  $\approx$  0.5 CPU). Baseline: `./cpu_work 8` fuera del cgroup.

**Predicción:** con cuota de 0.5 CPU espero que en  $\sim 8$  s de pared el workload (CPU-bound) complete **menos iteraciones** que el baseline (aproximadamente la mitad) y que en `cpu.stat` aumenten **`nr_throttled`** (períodos en que se agotó la cuota) y **`throttled_usec`** (tiempo de CPU denegado). No espero una razón exacta de 0.50.

**Evidencia (Capturas 5 y 6):**

| Medida                  | Baseline   | cpu-half   |
|-------------------------|------------|------------|
| <code>elapsed_s</code>  | 8.000748   | 8.005108   |
| <code>iterations</code> | 2037907456 | 1301020672 |

**cpu.stat del cgroup (inicial  $\rightarrow$  final):**

| Contador                    | Inicial | Final   | $\Delta$                                     |
|-----------------------------|---------|---------|----------------------------------------------|
| <code>usage_usec</code>     | 0       | 4012467 | <b>4012467</b> ( $\approx$ 4.01 s de CPU)    |
| <code>nr_throttled</code>   | 0       | 80      | <b>80</b>                                    |
| <code>throttled_usec</code> | 0       | 3991721 | <b>3991721</b> ( $\approx$ 3.99 s throttled) |

`nr_periods` final = 83. **razón de trabajo** =  $1301020672 / 2037907456 \approx 0.638$ .

```
>>> CAPTURA 5 - Baseline fuera del cgroup <<<
$ ./cpu_work 8
cpu_work pid=830897 duration_s=8.000
elapsed_s=8.000748
iterations=2037907456
checksum=15464561374366464549
```

**Captura obligatoria 5:** Baseline fuera del cgroup: `elapsed_s=8.000748`, `iterations=2037907456`.

```
>>> CAPTURA 6 (post) - Ejecucion limitada y cpu.stat final <<<
$ ./run-tn-cgroup.sh cpu-half ./cpu_work 8
cpu_work pid=831851 duration_s=8.000
elapsed_s=8.005108
iterations=1301020672
checksum=14919564302129043035

$ cat /sys/fs/cgroup/metricas-lab02/cpu-half/cpu.stat (final)
$ cat /sys/fs/cgroup/metricas-lab02/cpu-half/cpu.stat
usage_usec 4012467
user_usec 4012467
system_usec 0
nice_usec 0
nr_periods 83
nr_throttled 80
throttled_usec 3991721
nr_bursts 0
burst_usec 0

Incrementos (final - inicial):
usage_usec inicial=0 final=4012467 d=4012467
nr_throttled inicial=0 final=80 d=80
throttled_usec inicial=0 final=3991721 d=3991721

elapsed_s baseline=8.0007 cpu-half=8.0051
iterations baseline=2037907456 cpu-half=1301020672
razon de trabajo = iterations(cuota) / iterations(baseline) = 0.638
```

**Captura obligatoria 6:** Ejecución con cuota (`elapsed_s=8.005108`, `iterations=1301020672`) y `cpu.stat` final con `nr_throttled 80` y `throttled_usec 3991721`.

### 4.1. Contraste la predicción con las iteraciones observadas.

La predicción se cumplió cualitativamente: con cuota, el workload completó menos iteraciones (1301020672 vs 2037907456) en un `elapsed_s` casi idéntico. Cuantitativamente la razón ( $\approx 0.64$ ) quedó por encima del  $\approx 0.5$  nominal, lo que se explica por efectos del entorno (detallados en 4.4) y no invalida el mecanismo.

### 4.2. ¿Qué evidencia de `cpu.stat` permite afirmar el throttling? Sustente con los $\Delta$ .

Los incrementos en `cpu.stat`:  $\Delta nr\_throttled = 80$  (de 83 períodos, 80 fueron throttled: el cgroup exhaustó su cuota y el CFS bandwidth controller lo frenó) y  $\Delta throttled\_usec = 3991721 \mu s \approx 3.99$  s de CPU denegada. Son contadores exclusivos del controlador de cuota de cgroup v2: que se incrementen durante **esta** ejecución demuestra el throttling, y `usage_usec` ( $\approx 4.01$  s) cercano a la cuota ( $0.5 \times 8$  s) confirma que trabajó al máximo de lo asignado.

#### 4.3. ¿Por qué menos iteraciones en el mismo tiempo no es espera de I/O (Lab 1)?

Un proceso *throttled* está **runnable**: el scheduler le niega CPU periódicamente para respetar la cuota. Avanza menos en el mismo tiempo de pared **sin** bloquearse en I/O: no hay syscall de espera, ni estado de sleep, ni espera de disco. En el Lab 1, la espera de I/O se evidencia con estado dormido/esperando recurso, `iowait/%wa` y `time` con `user+sys << real`; aquí la evidencia son los contadores del scheduler (`nr_throttled`, `throttled_usec`), que no participan en una espera de I/O ordinaria.

#### 4.4. ¿Por qué no exigir razón exactamente 0.50?

`cpu.max` limita **tiempo de CPU**, no iteraciones; overhead de arranque/temporización, micro-usos de CPU del cgroup (kernel, page faults), granularidad del período (50/100 ms), ruido de otros procesos del host y jitter del cronometrado desplazan la razón (aquí, frecuencia de CPU turbo y carga del host la elevaron a  $\approx 0.64$ ). 0.5 es la fracción **máxima** asignable, no una equivalencia directa con trabajo completado.

## Investigación 5. Síntesis de mecanismos (10 puntos)

Tabla de síntesis:

| Mecanismo            | Qué modifica o controla                                                                                      | Evidencia principal utilizada                                                                     | Conclusión incorrecta a evitar                                              |
|----------------------|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <b>PID namespace</b> | La <b>vista</b> /identificación de los PID (qué procesos ve y cómo los numera); no asigna ni limita recursos | NSpid: 824974 1; PID 1 dentro vs 824974 desde el host                                             | "Dos PIDs $\Rightarrow$ dos procesos" / "el namespace limita CPU o memoria" |
| <b>memory.high</b>   | <b>Control</b> suave de memoria: presión/reclaim/throttle                                                    | high $\Delta = 125$ , allocation_complete 256 MiB, oom_kill $\Delta = 0$ , exit=0                 | "Es un límite duro de 128 MiB" / "termina al superarlo"                     |
| <b>memory.max</b>    | <b>Control</b> duro: con swap 0, alcanzar el tope $\rightarrow$ OOM killer del cgroup                        | max $\Delta = 35$ , oom $\Delta = 1$ , oom_kill $\Delta = 1$ , exit=137, último allocated_mib=120 | "El proceso falló por sí solo / reportó un error"                           |
| <b>cpu.max</b>       | <b>Control</b> de CPU: cuota de tiempo por período (throttling CFS)                                          | nr_throttled $\Delta = 80$ , throttled_usec $\Delta \approx 3.99$ s, razón $\approx 0.638$        | "Estaba esperando por I/O" / "tardó por carga normal"                       |

### 5.1. Diferencia entre aislamiento (namespaces) y control de recursos (cgroups).

Los **namespaces** dan *aislamiento*: cambian la **vista** del proceso sobre un recurso (Inv. 1: PID 1 dentro vs 824974 desde el host, NSpid: 824974 1), pero no acotan cuánto recurso puede usarse. Los **cgroups** son *control de recursos*: cuantifican y limitan el uso efectivo (Inv. 2/3: memory.events con high  $\Delta=125$  y max/oom/oom\_kill; Inv. 4: cpu.stat con nr\_throttled/throttled\_usec). Inv. 1 aporta evidencia de visibilidad; Inv. 2–4 de uso/consumo limitado.

### 5.2. Evidencia para distinguir espera, throttling y presión de memoria.

- **(a) Espera/bloqueo (Lab 1)**: estado del proceso (S/T/D), wchan, iowait/%wa (iostat/mpstat), time con user+sys  $\ll$  real; sin contadores de cgroup de por medio.
- **(b) Throttling de CPU (Lab 2)**: cpu.stat del cgroup del proceso con  $\Delta$  nr\_throttled y  $\Delta$  throttled\_usec  $> 0$ ; time con user+sys alto pero avanzando lento, sin estado de espera.
- **(c) Presión/límite de memoria (Lab 2)**: memory.events con  $\Delta$  en high/max/oom/oom\_kill, memory.current cerca de memory.max, swap con actividad, procesos terminados con 137.

### 5.3. ¿Por qué CPU baja o "tarda más" no basta para identificar el mecanismo?

"CPU baja" o "tarda más" son síntomas compartidos: un proceso **throttled** consume poca CPU por segundo y avanza poco (similar a una espera de I/O); uno bajo **presión de memoria** se frena por *reclaim* continuo (CPU baja o wall time con golpes); uno **bloqueado en I/O** muestra CPU baja por otra razón. Para atribuir el mecanismo se necesita la **fuentes específica** (contadores de cpu.stat, memory.events, memory.stat pgscan/pgsteal, estado/syscall del proceso) y el contexto (¿está en un cgroup con cuota? ¿cerca de memory.max? ¿esperando I/O?), como se hizo en 4.3 y 5.2.